

Servicio Web

Protocolo HTTP

Introducción

- HTTP es un protocolo base para la comunicación de datos en internet
- Se utiliza para transferir recursos, que pueden ser :
 - archivos,
 - el resultado de la ejecución de un programa
 - una consulta a una base de datos
 - la traducción automática de un documento
 - un formulario, etc.
- Es un protocolo “sin estado”, es decir, que no guarda ninguna información sobre conexiones anteriores.
 - Al finalizar la transacción todos los datos se pierden (*no existe el concepto de sesión !*).

Introducción

- Generalmente los servidores web escuchan las solicitudes de los clientes (navegadores) a través del [puerto 80](#).
- La versión actual de HTTP es la 2.0, y su especificación está en el [RFC-7540](#).
- HTTP dispone de una variante encriptada mediante SSL (*Secure Socket Layers*) llamada HTTPS (*HTTP Secure*), la cual se utiliza generalmente en sitios web que requieren cierto nivel de seguridad, como por ejemplo la banca electrónica, e-commerce.

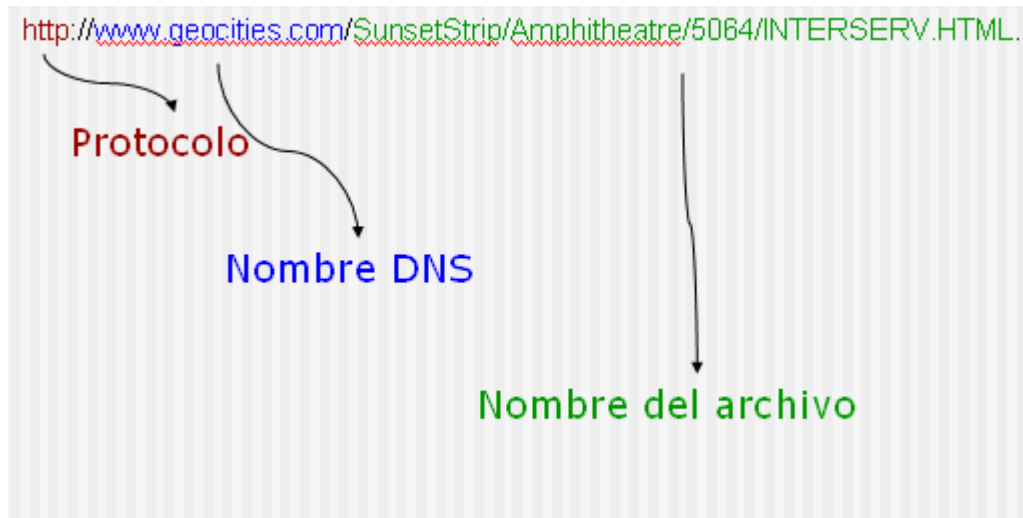
URL - Uniform Resource Locator

- Los recursos se identifican con un URL
- Este es un **identificador único** en la red para asociar el recurso dado.
- El formato del URL es:

protocolo://máquina:puerto/<archivo>

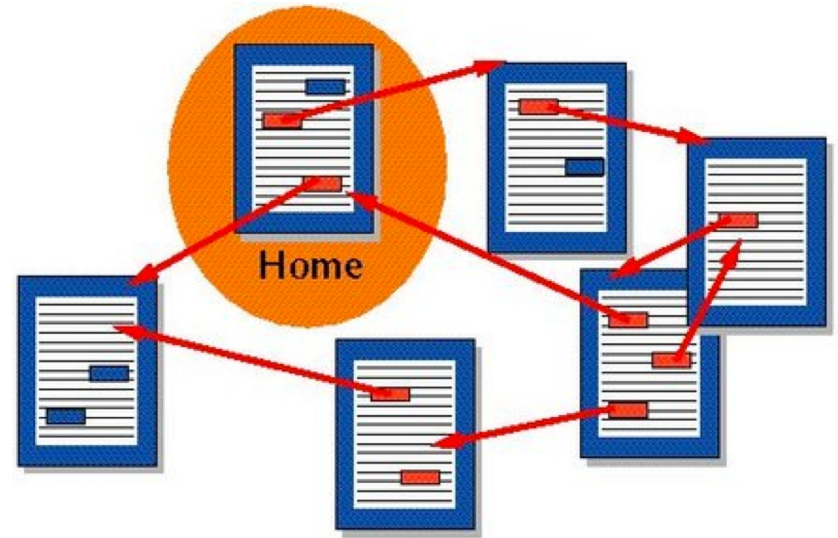
Para qué se usa un URL?

- Permiten contestar tres preguntas:
 - Cómo se llama el recurso (E.j. Una página)?
 - Dónde está el recurso?
 - Cómo se puede acceder al recurso?



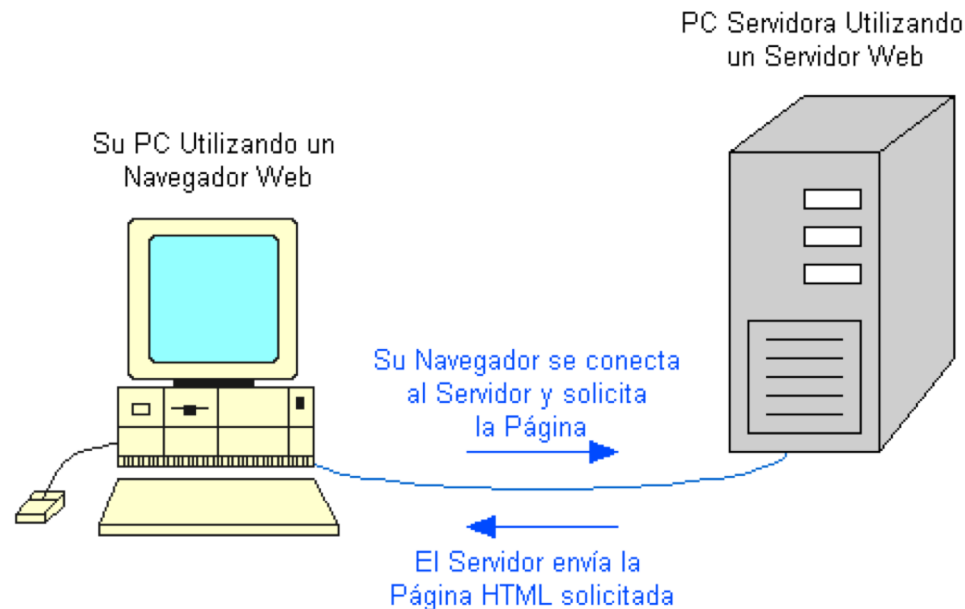
Hypertexts e Hyperlinks

- **Hypertext**: Texto estructurado que usa hyperlinks entre nodos que contienen texto
- **Hyperlink**: Cada pagina puede contener vínculos a otras páginas, y éstas a su vez a otras y así sucesivamente
- El protocolo HTTP transfiere hypertext



Arquitectura del Servicio Web

- Arquitectura Cliente – Servidor
- Las páginas se ven mediante un navegador
- El servidor almacena las páginas



Funciones del Cliente

- Es generalmente un navegador o browser
- Sus funciones incluyen:
 - Desplegar páginas Web
 - Atrapar los clic sobre la pagina desplegada
 - Seguir los hipervínculos

Una consulta típica desde el cliente

1. El Navegador determina la URL
2. El navegador se comunica con **el servidor de DNS** que traduce el nombre a dirección IP para que así se pueda conectar a la máquina destino
3. El Navegador crea una conexión TCP al servidor de la dirección IP en el puerto 80
4. El Navegador envía mensaje solicitando una página
5. El servidor envía el texto HTML de la página al Navegador.
6. Se libera la conexión
7. El Navegador lee los comandos HTML muestra la página en la pantalla.

MediaTypes

- MediaTypes: Comunicación Estandarizada de Datos en la Web
- Estándares que especifican el tipo de contenido de un recurso web.
- Ejemplos: text/plain, text/html, application/json, image/jpeg, audio/mpeg.
- En HTTP indican el tipo de contenido que se envía y se espera recibir en las respuestas HTTP
- Definidos por la IANA:

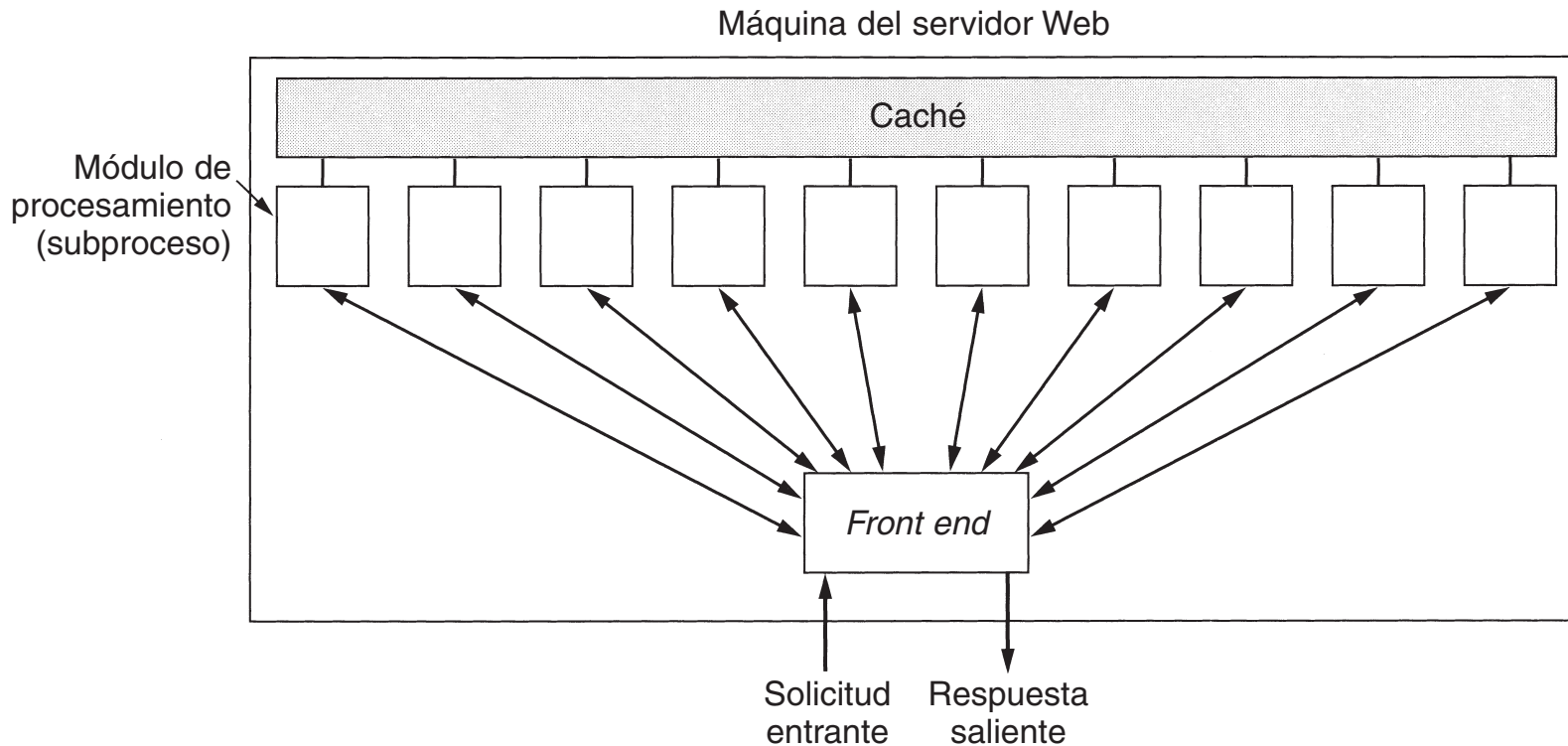
<https://www.iana.org/assignments/media-types/media-types.xhtml>

El Servidor

- Almacena los recursos (Ej. Páginas)
- Pasos para atender una petición:
 - Acepta una conexión TCP de un cliente
 - Obtiene el nombre del archivo solicitado
 - Obtiene el archivo del disco (o del caché)
 - Regresa el archivo al cliente
 - Libera la conexión TCP

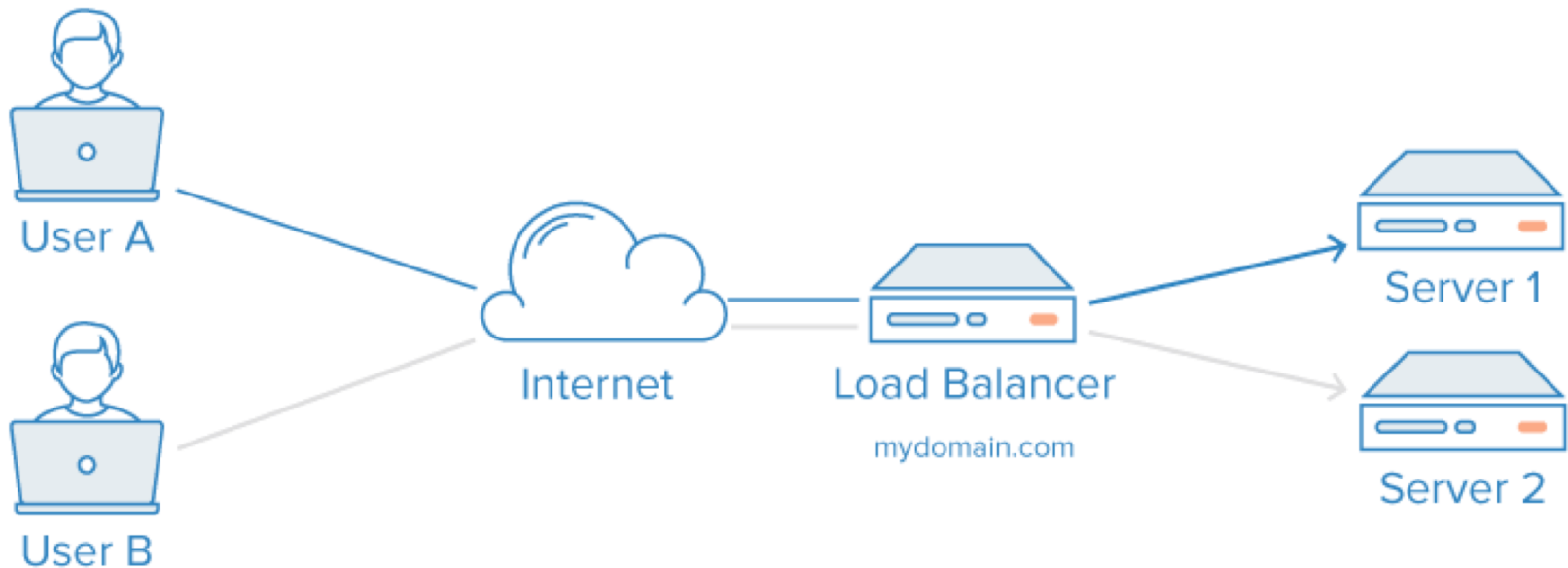
Atendiendo Múltiples Solicitudes

- Servidor web = Front end + múltiples subprocesos + cache.



Atendiendo Múltiples Solicitudes

- Balanceo de Carga y Escalamiento



<https://assets.digitalocean.com/blog/static/scaling-php/scenario.png>

HyperText Transfer Protocol (HTTP)

Generalidades del Protocolo

- Especifica mensajes y respuestas
 - Solicitud – mensaje ASCII
 - Respuesta – usando RFC 822
- La comunicación se realiza a través de una conexión TCP por el puerto 80
- HTTP1.0 Una vez se establecía la conexión, se enviaba una solicitud y se recibía la respuesta, después se liberaba la conexión
- HTTP1.1 Permite conexiones persistentes
- Versión actual: HTTP 2.0

Métodos de Petición

GET	/pet/{petId}	Find pet by ID
PUT	/pet	Update an existing pet
DELETE	/pet/{petId}	Deletes a pet
POST	/pet/{petId}/uploadImage	uploads an image

Fuente: <https://assertible.com/blog/7-http-methods-every-web-developer-should-know-and-how-to-test-them>



GET



PUT



POST



DELETE



Mensajes de respuesta

El servidor envía respuestas con los siguientes códigos de estado

Code	Meaning	Examples
1xx	Information	100 = server agrees to handle client's request
2xx	Success	200 = request succeeded; 204 = no content present
3xx	Redirection	301 = page moved; 304 = cached page still valid
4xx	Client error	403 = forbidden page; 404 = page not found
5xx	Server error	500 = internal server error; 503 = try again later

Fuente: Tanebaum, A. Redes de computadoras, 4 edición

Campos de Encabezado HTTP

- Son componentes de la sección de header de los mensajes de petición y respuesta
- Define los parámetros de una transacción HTTP
- Ejemplos:
 - Accept: text/plain
 - Content-Length: 348
- Mas en:

[LISTA DE ENCABEZADOS \(WIKIPEDIA\)](#)

Estructura de los mensajes HTTP

Petición

Request line *Method* *Request-URI* *Protocol version*
PUT /hr/ergonomics/posture.doc HTTP/1.1

Headers { Host: www.example.com:8080
Content-Length: 1234

Empty line

Body (optional) { Body must include the number of
characters specified in the content
length header...

Respuesta

Status line *Protocol version* *Status code* *Status description*
HTTP/1.1 200 OK

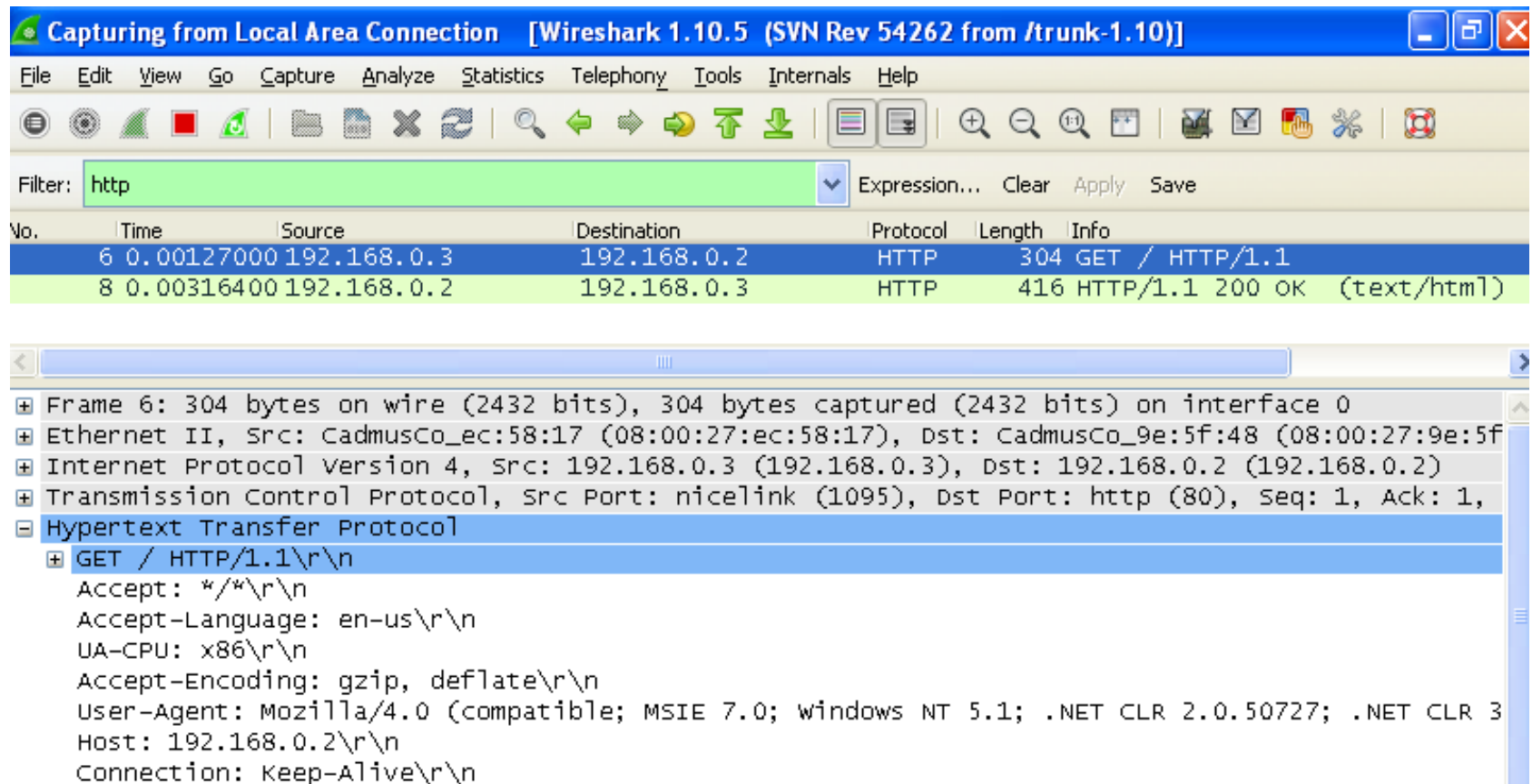
Headers { Date: Sun, 29 Jul 2001 15:24:17 GMT
Content-Length: 1234

Empty line

Body (optional) { Body must include the number of
characters specified in the content
length header...

Fuente: <http://flylib.com/books/en/2.320.1.30/1/>

Encabezado - Petición



The image shows the Wireshark 1.10.5 interface. The title bar reads "Capturing from Local Area Connection [Wireshark 1.10.5 (SVN Rev 54262 from /trunk-1.10)]". The menu bar includes File, Edit, View, Go, Capture, Analyze, Statistics, Telephony, Tools, Internals, and Help. The toolbar contains various icons for file operations, capture control, and analysis. The filter bar shows "http" selected. The packet list pane displays two packets:

No.	Time	Source	Destination	Protocol	Length	Info
6	0.00127000	192.168.0.3	192.168.0.2	HTTP	304	GET / HTTP/1.1
8	0.00316400	192.168.0.2	192.168.0.3	HTTP	416	HTTP/1.1 200 OK (text/html)

The packet details pane for the selected packet (Frame 6) shows the following structure:

- Frame 6: 304 bytes on wire (2432 bits), 304 bytes captured (2432 bits) on interface 0
- Ethernet II, Src: CadmusCo_ec:58:17 (08:00:27:ec:58:17), Dst: CadmusCo_9e:5f:48 (08:00:27:9e:5f)
- Internet Protocol Version 4, Src: 192.168.0.3 (192.168.0.3), Dst: 192.168.0.2 (192.168.0.2)
- Transmission Control Protocol, Src Port: nicelink (1095), Dst Port: http (80), Seq: 1, Ack: 1,
- Hypertext Transfer Protocol**
 - GET / HTTP/1.1\r\n**
 - Accept: */*\r\n
 - Accept-Language: en-us\r\n
 - UA-CPU: x86\r\n
 - Accept-Encoding: gzip, deflate\r\n
 - User-Agent: Mozilla/4.0 (compatible; MSIE 7.0; windows NT 5.1; .NET CLR 2.0.50727; .NET CLR 3
 - Host: 192.168.0.2\r\n
 - Connection: Keep-Alive\r\n

Encabezado - Respuesta

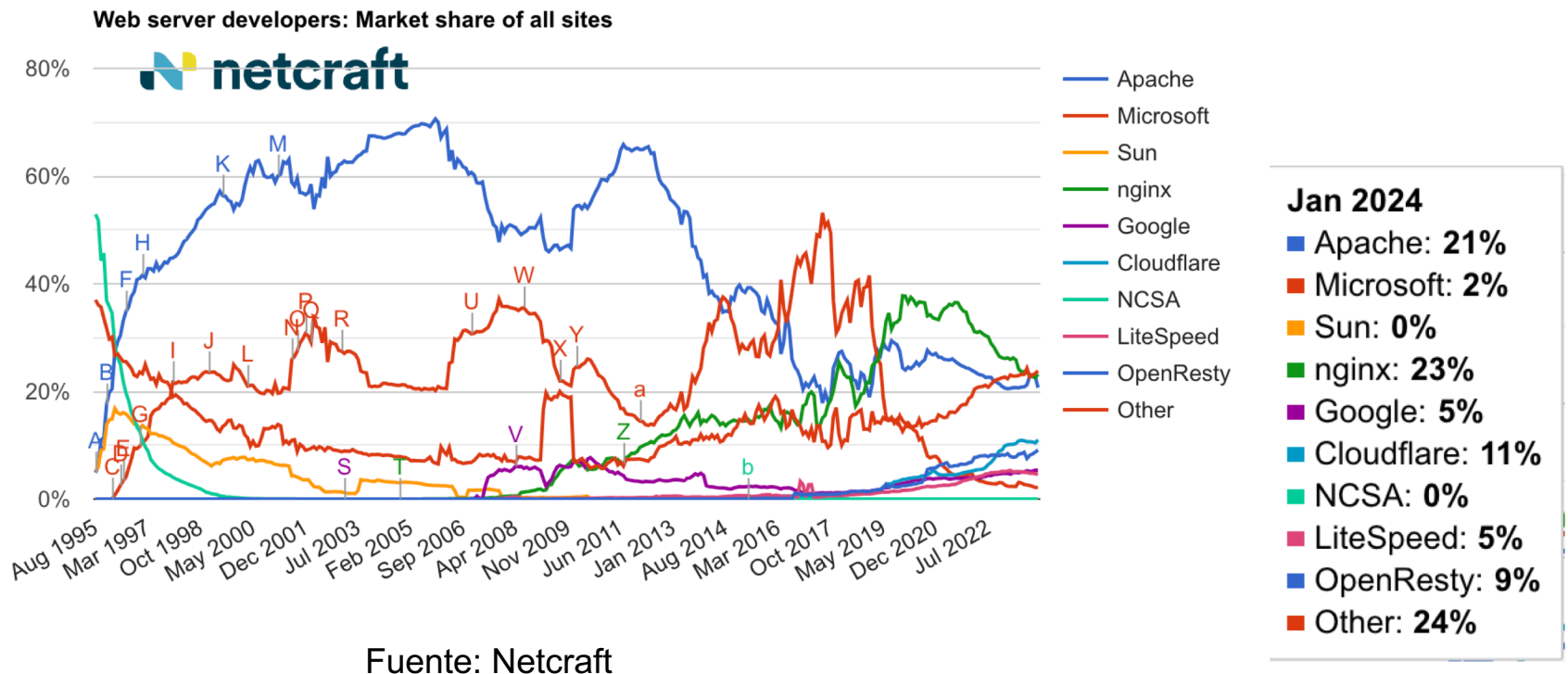
The image shows the Wireshark 1.10.5 interface. The title bar reads "Capturing from Local Area Connection [Wireshark 1.10.5 (SVN Rev 54262 from /trunk-1.10)]". The menu bar includes File, Edit, View, Go, Capture, Analyze, Statistics, Telephony, Tools, Internals, and Help. The toolbar contains various icons for packet capture and analysis. The filter bar shows "Filter: http". The packet list pane displays two packets:

No.	Time	Source	Destination	Protocol	Length	Info
6	0.00127000	192.168.0.3	192.168.0.2	HTTP	304	GET / HTTP/1.1
8	0.00316400	192.168.0.2	192.168.0.3	HTTP	416	HTTP/1.1 200 OK (text/html)

The packet details pane for packet 8 shows the following structure:

- Transmission Control Protocol, Src Port: http (80), Dst Port: nicelink (1095), Seq: 1, Ack: 251
- Hypertext Transfer Protocol
 - HTTP/1.1 200 OK\r\n
 - Date: Thu, 13 Feb 2014 14:40:13 GMT\r\n
 - Server: Apache/2.2.15 (CentOS)\r\n
 - Last-Modified: Thu, 13 Feb 2014 14:39:53 GMT\r\n
 - ETag: "1f9d9-5f-4f24aa99b51ab"\r\n
 - Accept-Ranges: bytes\r\n
 - Content-Length: 95\r\n
 - [Content length: 95]
 - Connection: close\r\n
 - Content-Type: text/html; charset=UTF-8\r\n
 - \r\n
 - [HTTP response 1/1]
 - [Time since request: 0.001894000 seconds]

Servidores web en el mercado (2024)



Apache

- Servidor web de código fuente abierto distribuido bajo licencia GPL (General Public License), multiplataforma (Unix, Linux, Windows, Solaris, etc)
- Apache implementa el protocolo HTTP/1.1, HTTP/2 y el concepto de “virtual hosting”.
- Su desarrollo inició en 1995 y se basó en el código fuente del servidor web de la NCSA (National Center for Supercomputing Applications).
 - De allí se deriva su nombre - un conjunto de parches al servidor web de NCSA “a patchy server” (un servidor parchado).

Características de Apache

- Mensajes de error altamente configurables
- Bases de datos de autenticación
- Integración con paquetes de software como MySQL, Postgress, PHP
- Virtual hosting
- SW Libre desarrollado y mantenido por la comunidad de programadores
- No posee una interfaz gráfica para su configuración

Estructura de Apache

- Es modular: El servidor Apache consta de una sección “núcleo” (core) y las demás funcionalidades básicas son provistas por módulos
- En ubuntu el directorio de módulos es:
</usr/lib/apache2/modules/>
- Para ver los módulos cargados usar:

```
$ apachectl -M
```

```
Loaded Modules:
```

```
core_module (static)
```

```
so_module (static)
```

```
watchdog_module (static)
```

```
http_module (static)
```

```
...
```

Algunos módulos disponibles

- mod_ssl- Comunicaciones seguras usando TLS.
- mod_auth_ldap - Permite autenticar usuarios contra un servidor LDAP (Lightweight Directory Access Protocol).
- mod_proxy_ajp - Conector para enlazar con el servidor Jakarta Tomcat de páginas dinámicas en Java (servlets y JSP).
- mod_perl - Páginas dinámicas en Perl.
- mod_php - Páginas dinámicas en PHP.
- mod_python - Páginas dinámicas en Python.
- mod_aspdotnet - Páginas dinámicas en .NET_de_Microsoft
- mod_http2 – soporte para HTTP/2

Instalación de Apache en Ubuntu

- Verificar si ya esta instalado el proceso del servicio

```
apt list apache2 --installed
```

- Para instalarlo

```
sudo apt install apache2
```

Archivos de configuración de Apache

- Los archivos de configuración en Ubuntu están en el directorio [/etc/apache2/](#)
- Entre los archivos de configuración básicos están:
 - [apache2.conf](#) : principal archivo de configuración del servidor web.
 - [.htaccess](#) : permite restringir el acceso a cada directorio del servidor web.

Configuración

Apache permite configurar:

- Parámetros globales
- Directivas de Funcionamiento
- Host Virtuales

Parámetros Globales

Todos los parámetros que se establecen dentro de esta sección son globales para el funcionamiento del servidor.

- **ServerRoot**: especifica la ubicación del directorio raíz donde se encuentra instalado el Apache (apache2.conf).
- **KeepAlive**: especifica si se utilizarán conexiones persistentes (apache2.conf).
- **Listen**: esta directiva permite especificar que puerto se utilizará para atender las peticiones (ports.conf).
- **LoadModule**: Directiva que sirve para cargar módulos que incluyen distintas funcionalidades (mods-available).

Directivas de Funcionamiento

- **ServerAdmin:** especifica la dirección de correo electrónico del administrador (sites-available).
- **ServerName:** especifica el nombre y el puerto que el servidor utiliza para identificarse (apache2.conf).
 - Si el servidor no tiene un nombre registrado en las DNS, se recomienda poner su dirección IP.

apache2.conf– Directivas de Funcionamiento

- **DocumentRoot:** la carpeta raíz que se ubica en el servidor, desde la que se servirán los documentos (sites-available).
 - Si se cambia este directorio por otro, es muy importante que se ponga el nuevo valor, no solo en esta línea, sino también en la sección <Directory> en la que se establecen los parámetros de configuración de este directorio.
- **DirectoryIndex:** especifica el fichero por defecto que buscará en cada directorio (mods-available/dir.conf).
 - En caso de que no se especifique ninguno, por defecto es index.html.

Host Virtuales (sites-available)

```
root@localhost:/etc/httpd/conf
Archivo  Editar  Ver  Terminal  Solapas  Ayuda
#
# Use name-based virtual hosting.
#
#NameVirtualHost *:80
#
# NOTE: NameVirtualHost cannot be used without a port specifier
# (e.g. :80) if mod_ssl is being used, due to the nature of the
# SSL protocol.
#
#
# VirtualHost example:
# Almost any Apache directive may go into a VirtualHost container.
# The first VirtualHost section is used for requests without a known
# server name.
#
#<VirtualHost *:80>
#    ServerAdmin webmaster@dummy-host.example.com
#    DocumentRoot /www/docs/dummy-host.example.com
#    ServerName dummy-host.example.com
#    ErrorLog logs/dummy-host.example.com-error_log
#    CustomLog logs/dummy-host.example.com-access_log common
#</VirtualHost>
```

Administrar el servicio

```
systemctl status apache2
```

```
systemctl start apache2
```

```
systemctl stop apache2
```

```
systemctl restart apache2
```

Pruebas desde el Cliente

Prueba 1

- Prueba con dirección IP

Prueba 2

- Configurar DNS
- Prueba con nombre del servidor

Opciones de seguridad

Se puede restringir el acceso a ciertos recursos (archivos) de 3 maneras:

- Configurando la **opción Directory** en el archivo `httpd.conf`.
- Utilizando un **archivo de claves (password file)** para autenticar el acceso de los usuarios.
 - Si un usuario no se identifica correctamente el servidor web devuelve el error 401 – acceso no autorizado.
- Usando **archivos .htaccess** , que contengan las directivas de restricción del acceso a los recursos (no requiere reinicio del servidor).

Seguridad usando la opción Directory en `/etc/apache2/apache2.conf`

```
<Directory /var/www/>  
    Order allow,deny  
    Allow from all  
    Deny from 192.168.50.2  
</Directory>
```

Seguridad usando archivos de claves

- Para crear el archivo de claves se usa el comando `htpasswd`:

`htpasswd -c /etc/apache2/claves juan`

- Crea un archivo de claves con un registro para el nuevo usuario “juan”, y luego solicita la clave.

- Para adicionar otros usuarios al archivo de claves creado se usa el comando de la siguiente manera:

`htpasswd /etc/apache2/claves <nombre_usuario>`

- Nota: el archivo de claves debe estar ubicado en un directorio donde solo tenga permisos el administrador (root).

Seguridad usando archivos de claves

- Se debe agregar en [apache2.conf](#) la configuracion del directorio protegido

```
<Directory "/var/www/html/protegido">  
    AuthType Basic  
    AuthName "Directorio protegido"  
    AuthUserFile /etc/apache2/claves  
    Require valid-user  
</Directory>
```

Seguridad usando archivos .htaccess

- Los archivos .htaccess son pequeños archivos de texto invisibles que permiten controlar algunas funciones de configuración en un servidor Apache.
 - Para poder utilizar archivos .htaccess el servidor debe tener activada la directiva AllowOverride y, dentro de ella, definidas las variables que nos va a permitir modificar.
 - Cuando la directiva AllowOverride está activa el servidor realiza una búsqueda por todos los directorios hasta encontrar el archivo .htaccess correspondiente y, después, en consecuencia muestra la página al usuario. Si no encuentra ningún .htaccess aplicará las directivas establecidas en la configuración de Apache.

Seguridad usando archivos .htaccess

- Alrededor de la linea ~170 editar apache2.conf para activar el AllowOverride

```
<Directory "/var/www/">
```

```
    AllowOverride All
```

```
    ...
```

```
</Directory>
```

Ejemplo de archivo .htaccess

#Define el tipo de autenticación.

AuthType Basic

#Define la ubicación de los archivos de password
creados con htpasswd

AuthUserFile /etc/apache2/claves

#Define el nombre de los recursos.

AuthName "Directorio Privado"

#permiso a todos los usuarios.

Require valid-user

Ejemplo de archivo .htaccess

Cree un directorio protegido2 en la ruta
[/var/www/html/protegido2](#)

Dentro de protegido2 cree un archivo .htaccess con
el contenido de la diapositiva anterior

Pruebe que funciona el acceso restringido

Virtual Hosting

- Apache permite administrar más de un sitio en un solo “host” de una manera sencilla y eficiente.
 - Se pueden recibir peticiones para varios servidores web en una misma máquina, los cuales se conocen como “host virtuales”.
 - Un host virtual se puede configurar usando la [directiva <VirtualHost>](#) del archivo httpd.conf .
- Apache permite 2 tipos de virtual hosting:
 - 1) Basado en nombres, si se tiene una sola IP.
 - 2) Basado en IP, cuando se tienen varias IPs.
- Para su correcto funcionamiento se deben configurar las zonas correspondientes en el DNS

Virtual hosting basado en nombres

Crear un archivo de configuración por cada sitio en [sites-available](#)

Ejemplo: **/etc/apache2/sites-available/servicios.com.conf**

```
<VirtualHost *:80>  
    ServerName www.servicios.com  
    DocumentRoot /var/www/html  
</VirtualHost>
```

Ejemplo: **/etc/apache2/sites-available/otrodominio.com.conf**

```
<VirtualHost *:80>  
    ServerName www.otrodominio.com  
    DocumentRoot /var/www/otrodominio  
</VirtualHost>
```

NOTA: Asegurarse que el dns este configurado para resolver [www.servicios.com](#) y [www.otrodominio.com](#)

Activando los sitios virtuales

Habilite los sitios virtuales, deshabilite el sitio por defecto y recargue :

```
a2ensite otrodominio.com.conf  
a2ensite servicios.com.conf  
a2dissite 000-default.conf  
systemctl reload apache2
```

Al ejecutar los anteriores comandos, los sitios son creados en </etc/apache2/sites-enabled/>

Pruebe la configuración:
`sudo apache2ctl configtest`

Reinicie el servicio:
`systemctl restart apache2`
`sudo systemctl status apache2`

Probando los sitios virtuales

Desde el cliente, ejecute:

```
curl www.servicios.com
```

```
curl www.otrodominio.com
```

Virtual hosting basado en IP

En caso de disponer de más de una interfaz de red en el servidor, se puede crear un archivo de configuración por cada sitio en sites-available

```
<VirtualHost 192.168.1.15>  
    ServerName www.midominio.com  
    DocumentRoot /var/www/midominio  
</VirtualHost>  
  
<VirtualHost 192.168.1.16>  
    ServerName www.otrodomnio.com  
    DocumentRoot /var/www/otrodominio  
</VirtualHost>
```

Instalar módulo userDir para página personal

- Agregue el módulo userDir

```
sudo a2enmod userdir
```

- Valide el archivo `/etc/apache2/mods-available/userdir.conf` que debe tener un contenido como este:

```
UserDir public_html
```

```
UserDir disabled root
```

```
<Directory /home/*/public_html>
```

```
    AllowOverride FileInfo AuthConfig Limit Indexes
```

```
    Options MultiViews Indexes SymLinksIfOwnerMatch
```

```
    IncludesNoExec
```

```
    Require method GET POST OPTIONS
```

```
</Directory>
```

Crear una página personal

- Cree un usuario

```
useradd usuario
```

```
passwd usuario
```

- Cree un directorio public_html dentro del home del usuario

```
mkdir /home/usuario/public_html
```

- Establezca permisos

```
chmod 711 /home/usuario
```

```
chmod 755 /home/usuario/public_html
```

- Cree Pagina [index.html](#) dentro de public_html

- Reinicie el servicio

```
sudo systemctl restart apache2.service
```

- Desde un navegador, acceda usando

```
http://<ip_del_servidor>/~usuario
```

Referencias

- Configuración e instalación de Apache.
<https://ubuntu.com/server/docs/how-to-install-apache2>
- Configurar virtual host en apache
<https://www.digitalocean.com/community/tutorials/how-to-set-up-apache-virtual-hosts-on-ubuntu-20-04>
- Configurar página personal. <https://geekrewind.com/setup-apache2-userdir-module-on-ubuntu-16-04-lts-servers/>
- TANENBAUM, Andrew S. “Redes de Computadores”. Cuarta Edición. Prentice Hall, 2003.
- GARCIA, Alexander. Notas de Clase. Servicios de Red – Unidad 2-HTTP
- SOLARTE, Zeida. Notas de Clase. Servicios Multimedia.
- Pagina oficial de Apache. <http://www.apache.org/>